

MLOPS: LAB3

CONTINUOUS TRAINING: APP-IRIS-CT

Carlos Molina Martínez
Curso 2025-26

1. Introducción

El presente documento recoge los resultados, reflexiones y decisiones de diseño derivados de la práctica LAB 3 enfocada en arquitecturas de Continuous Training (CT). Mientras que las implementaciones de MLOps de nivel básico se limitan a desplegar modelos estáticos bajo el paradigma *deploy once, serve forever*, los entornos de producción reales requieren sistemas que se adapten a la evolución continua de los datos.

El objetivo principal de este trabajo ha sido evolucionar un servicio de inferencia hacia una API REST dinámica, capaz de combinar inferencia y reentrenamiento automatizado. Para lograrlo, se ha implementado un sistema de control de versiones para los artefactos de Machine Learning y un *quality gate* que evalúa cada nuevo modelo, activándolo únicamente si logra mejorar las métricas de calidad del modelo en producción. En las siguientes secciones se da respuesta a las cuestiones analíticas planteadas sobre reproducibilidad, auditoría de modelos rechazados y el diseño de políticas de activación configurables, complementando el desarrollo técnico realizado en el código adjunto.

2. Ejercicios

2.1. Ejercicio 1: Reproducir el workflow

En este primer apartado, se ha procedido a inicializar el servidor de la API REST y a ejecutar el script de demostración (`demo_ct.py`) utilizando el flag `--reset`. Este comando restaura el modelo base y ejecuta automáticamente un flujo completo de *Continuous Training* compuesto por cuatro pasos. A continuación, se adjuntan las evidencias solicitadas: la captura con la salida completa por terminal del proceso, así como la respuesta en formato JSON obtenida al consultar el endpoint `/model/info` tras finalizar la

ejecución.

```
DEMO: Iris Continuous Training API

Host: http://localhost:8000

RESET - Restaurando modelo base

[✓] Historial eliminado. Modelo base restaurado.

PASO 0 - Health check

[✓] Servicio activo. Modelo activo: v1.0-base

PASO 0b - Info modelo base

[✓] Versión activa : v1.0-base
[✓] Entrenado el   : 2026-03-10T11:54:10.376269Z
[✓] Accuracy      : 0.9667
[✓] Nº muestras   : 120
[✓] Algoritmo     : LogisticRegression

[i] Historial de versiones (1 entradas):
● v1.0-base | acc=0.9667 | 2026-03-10T11:54:10 | -

PASO 1 - Predicción con el modelo base

[✓] Predicción: clase 0 (setosa), versión modelo: v1.0-base

PASO 2 - Reentrenamiento con muestras CORRECTAS
```

```
[i] Enviando 20 muestras bien etiquetadas...
[✓] Nuevo modelo ACTIVADO → versión: v2.0-9dd867
[✓] Accuracy nuevo: 1.0000 | Anterior: 0.9667
[i] Nuevo modelo activado. Accuracy 1.0000 >= anterior (0.9667)

PASO 3 - Reentrenamiento con muestras RUIDOSAS (etiquetas incorrectas)

[i] Enviando 10 muestras con etiquetas erróneas...
[✓] Modelo RECHAZADO como esperado. Accuracy 0.6000 < anterior 1.0000
[i] Modelo NO activado. Accuracy 0.6000 < anterior (1.0000). El modelo activo se mantiene sin cambios.

PASO 4 - Predicción con el modelo actualizado

[✓] Predicción: clase 0 (setosa), versión modelo: v3.0-72f288

RESUMEN FINAL - Historial de versiones

[✓] Versión activa : v2.0-9dd867
[✓] Entrenado el   : 2026-03-10T11:54:19.343636Z
[✓] Accuracy      : 1.0000
[✓] Nº muestras   : 20
[✓] Algoritmo     : LogisticRegression

[i] Historial de versiones (3 entradas):
● v1.0-base | acc=0.9667 | 2026-03-10T11:54:10 | -
● v2.0-9dd867 | acc=1.0000 | 2026-03-10T11:54:19 | activado
● v3.0-72f288 | acc=0.6000 | 2026-03-10T11:54:21 | rechazado

[✓] Demo completado. Abre http://localhost:8000/docs para explorar la API.
```

```

1  {
2  {
3    "version": "v1.0-base",
4    "trained_at": "2026-03-10T11:54:10.376269Z",
5    "accuracy": 0.9667,
6    "n_training_samples": 120,
7    "algorithm": "LogisticRegression",
8    "source": "bootstrap (iris dataset completo)"
9  },
10 {
11   "version": "v2.0-9dd867",
12   "trained_at": "2026-03-10T11:54:19.343636Z",
13   "accuracy": 1.0,
14   "n_training_samples": 20,
15   "algorithm": "LogisticRegression",
16   "source": "desde cero (20 muestras)",
17   "eval_note": "validaci\u00f3n con 4 muestras",
18   "status": "activado",
19   "activated": true
20 },
21 {
22   "version": "v3.0-72f288",
23   "trained_at": "2026-03-10T11:54:21.473964Z",
24   "accuracy": 0.6,
25   "n_training_samples": 10,
26   "algorithm": "LogisticRegression",
27   "source": "desde cero (10 muestras)",
28   "eval_note": "evaluaci\u00f3n en train (dataset peque\u00f1o, < 20 muestras)",
29   "status": "rechazado",
30   "activated": false
31 }
32 ]

```

2.2.Ejercicio 2: Explorar el historial

Tras analizar el fichero `models/training_history.json` generado durante la ejecución de la demo, se da respuesta a las cuestiones planteadas:

- ¿Cuántas versiones se han registrado?

Se han registrado un total de 3 versiones en el historial, coincidiendo exactamente con los pasos del entrenamiento observados en la salida de la terminal.

- ¿Qué versiones fueron activadas y cuales rechazadas?

De las tres entradas registradas, las dos primeras fueron activadas (`"activated": true`). Estas corresponden a la versión inicial `v1.0-base` (creada por defecto al arrancar, con un *accuracy* de 0.9667) y a la versión `v2.0-9dd867` (que logró un *accuracy* perfecto de 1.0 tras el reentrenamiento con muestras correctas). Por el contrario, la tercera versión (`v3.0-72f288`) fue rechazada (`"activated": false`).

- ¿Por qué el modelo entrenado con muestras ruidosas fue rechazado?

La tercera versión fue rechazada debido a que la API implementa un *gate* de calidad automatizado. El modelo `v3.0-72f288` fue entrenado con muestras ruidosas, lo que provocó que su rendimiento cayera a un *accuracy* de 0.6. Dado

que la regla de negocio establece que solo se activa un modelo si se cumple la condición $\text{accuracy_nuevo} \geq \text{accuracy_anterior}$, el sistema rechazó la nueva versión ($0.6 < 1.0$) y mantuvo activo el modelo anterior para proteger el entorno de producción.

2.3. Ejercicio 3: Entrenamiento incremental vs desde cero.

Al realizar dos llamadas consecutivas al endpoint /train utilizando el mismo conjunto de 10 muestras, pero variando el parámetro de configuración, se han registrado los siguientes resultados en el historial:

- Entrenamiento incremental (`retrain_from_scratch: false`): Se generó la versión v4.0-43223e obteniendo un *accuracy* de 1.0. Al acumular las 10 muestras nuevas a las 20 preexistentes, el sistema alcanzó un dataset de 30 muestras. Esto permitió al algoritmo realizar una partición adecuada y evaluar el modelo sobre un conjunto de validación real (6 muestras), demostrando una alta capacidad de generalización y un rendimiento óptimo.
- Entrenamiento desde cero (`retrain_from_scratch: true`): Se generó la versión v5.0-6aee35 obteniendo también un *accuracy* de 1.0. Sin embargo, este resultado es un artefacto estadístico. Al descartar el histórico y entrenar exclusivamente con las 10 muestras enviadas, el sistema detecta que el dataset es demasiado pequeño (< 20 muestras) y evalúa el modelo sobre sus propios datos de entrenamiento. El resultado perfecto (1.0) es consecuencia directa de un sobreajuste (*overfitting*), ya que el modelo simplemente memoriza las muestras sin capacidad real de generalizar ante datos desconocidos.

Aunque numéricamente no se observan diferencias en la métrica final (ambos devuelven 1.0), el comportamiento interno es radicalmente distinto. El modelo incremental es robusto y fiable, mientras que el entrenado desde cero con pocas muestras carece de validez para un entorno de producción debido al sobreajuste.

3. Preguntas

3.1. Pregunta 1

Cuando arrancas el servidor por primera vez sin ningún modelo previo, el sistema crea automáticamente un modelo base mediante el proceso de *bootstrap*. Si detuvieras el servidor, borraras la carpeta `models/` y lo volvieras a arrancar, ¿obtendrías exactamente el mismo modelo con el mismo *accuracy*? Razona por qué sí o por qué no, y señala qué línea o parámetro del código garantiza o impide esa reproducibilidad.

Sí, si se detuviera el servidor, se borrara la carpeta `models/` y se volviera a arrancar, se obtendría exactamente el mismo modelo con idéntico *accuracy* (0.9667).

Esto es debido a que en el código se garantiza la reproducibilidad del proceso de entrenamiento mediante la fijación de la semilla aleatoria (random seed). Existen específicamente dos líneas de código en la función `bootstrap_model()` responsables de este comportamiento:

1. En la partición del dataset: `train_test_split(..., random_state=42, ...)`. Este parámetro obliga a que la división aleatoria entre datos de entrenamiento (80%) y validación (20%) sea idéntica en cada ejecución.

```
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

2. En la inicialización del modelo: `LogisticRegression(..., random_state=42)`. Este parámetro asegura que el proceso iterativo interno del algoritmo comience siempre desde las mismas condiciones iniciales.

```
clf = LogisticRegression(max_iter=200, random_state=42)
clf.fit(X_train, y_train)
```

3.2. Pregunta 2

El endpoint `/train` exige un mínimo de 5 muestras por request y además comprueba que haya al menos 2 clases distintas en el dataset de entrenamiento. ¿Qué pasaría si enviaras 10 muestras perfectamente etiquetadas pero todas de la clase setosa? ¿El modelo se entrenaría? ¿Se activaría? Describe paso a paso qué devolvería la API y por qué tiene sentido esa restricción.

Si se enviaran 10 muestras perfectamente etiquetadas pero todas pertenecientes a una única clase (setosa), el sistema abortaría el proceso de entrenamiento de forma inmediata. La API devolvería un error HTTP 422 Unprocessable Entity junto con el mensaje explícito: *"El dataset de entrenamiento debe contener al menos 2 clases distintas"*. Por consiguiente, el modelo ni se entrenaría ni llegaría a evaluarse, y mucho menos se activaría.

Un modelo predictivo, como la Regresión Logística utilizada en este sistema, funciona trazando fronteras de decisión o "límites" entre diferentes categorías. Si se le proporcionan datos de una única clase, el algoritmo carece de información para aprender a distinguir. De permitirse este entrenamiento, el modelo sufriría un colapso: aprendería a predecir "setosa" para el 100% de las flores que procesara en el futuro, volviéndose completamente inútil para identificar las especies *versicolor* o *virginica*.

3.3. Pregunta 3

Ejecuta la demo con `--reset` y observa la secuencia de versiones en el historial final. El modelo entrenado con muestras ruidosas aparece en el historial aunque no haya sido activado. ¿Por qué crees que se diseñó así, registrando también los modelos rechazados? ¿Qué información útil proporciona ese registro que no tendríamos si solo guardáramos los modelos activados?

Creo que se diseñó así porque es importante mantener un registro completo de todas las acciones que se intentan llevar a cabo en el sistema, documentando qué modelos candidatos se generan, qué *accuracy* obtienen y el motivo exacto por el que no logran activarse.

Es necesario para tener una trazabilidad sobre que conjunto de datos de entrenamiento se provocó la degradación del modelo, facilitando así la detección de problemas o anomalías.

Si el historial mostrara una racha continuada de modelos rechazados, actuaría como una alerta temprana de que la calidad de los datos entrantes en el flujo de producción se ha deteriorado.

3.4. Pregunta 4

El gate de calidad usa la regla `accuracy_nuevo >= accuracy_anterior` para decidir si activar el modelo. Con esta regla, un modelo nuevo que obtiene exactamente el mismo *accuracy* que el anterior sí se activa. ¿Crees que eso es correcto o debería usarse `>` estricto? Argumenta tu respuesta pensando en qué ocurre cuando el dataset de entrenamiento crece con muestras de buena calidad pero el *accuracy* se estabiliza cerca del máximo

Sí, considero que es correcto usar el operador `>=` en lugar del `>` estricto por varios motivos:

1. Aporta más robustez: Al introducir y acumular datos nuevos, el modelo aprende de una mayor cantidad y variedad de casos. Esto hace que el sistema sea más robusto y generalice mejor, aunque la métrica de *accuracy* ya haya alcanzado su límite y no pueda subir más.
2. Adaptación a nuevos patrones: Los datos recientes pueden contener información más actualizada. Aunque asimilar estos nuevos datos no se refleje en una mejora del *accuracy* a corto plazo, a la larga es vital para evitar que el modelo se quede obsoleto si los datos del mundo real empiezan a cambiar.

4. Ejercicio de programación

4.1. Fase de razonamiento previo

Considera los siguientes tres escenarios hipotéticos. Elige y justifica qué política aplicarías en cada uno:

- Escenario 1: un modelo de detección de fraude bancario donde los falsos negativos (fraude no detectado) son mucho más costosos que los falsos positivos.
- Escenario 2: un modelo de recomendación de películas donde cualquier mejora marginal en *accuracy* es bienvenida porque el coste de un error es bajo.
- Escenario 3: un modelo médico de diagnóstico donde la estabilidad y previsibilidad del comportamiento importan tanto como la precisión.

A continuación, se justifica la elección de la política óptima para cada caso de uso, basándonos en los tres modos propuestos para el desarrollo (*any_improvement*, *min_delta* y *per_class_f1*) :

Escenario 1:

Se aplicaría la política *per_class_f1* enfocada en la clase "fraude". En este contexto, es preferible soportar un aumento de falsos positivos (alertas de fraude que no lo son) antes que permitir falsos negativos (dejar pasar un fraude real). Si el *accuracy* global subiera pero la capacidad de detectar la clase minoritaria bajara, el modelo debería rechazarse. Usar el F1-score sobre esa clase específica garantiza que solo se active si mejora realmente en lo que importa al negocio.

Escenario 2:

Se aplicaría la política *any_improvement* (el operador $\geq$ actual). Dado que el coste de cometer un error al recomendar es muy bajo, cualquier mínima subida en el rendimiento es bienvenida. Además, aceptar modelos que igualan la métrica es beneficioso, ya que permite que el sistema integre de forma implícita y continua los cambios graduales en los gustos temporales de los consumidores sin ser penalizado.

Escenario 3:

Se aplicaría la política *min_delta*. En el ámbito clínico, la estabilidad, previsibilidad e interpretabilidad del sistema son críticas. Implementar un modelo nuevo conlleva riesgos y costes operativos altos. Por tanto, no se debe alterar el sistema por fluctuaciones marginales. Exigir un porcentaje de mejora mínimo (por ejemplo, $> 3\%$) asegura que solo se reemplace el modelo en producción cuando la nueva versión demuestre ser indiscutible y significativamente superior, justificando el cambio.

4.2.Pseudocódigo

CLASE ActivationPolicy:

ATRIBUTO política (Puede ser "any_improvement", "min_delta", "per_class_f1")

ATRIBUTO min_delta (Porcentaje de mejora mínima (ej. 0.02 para 2%))

ATRIBUTO target_class (Clase a evaluar en per_class_f1)

MÉTODO INICIALIZAR (tipo_política, delta_opcional = 0, clase_opcional =
“”):

ESTE.politica = tipo_política

ESTE.min_delta = delta_opcional

ESTE.target_class = clase_opcional

MÉTODO evaluar_modelo (metricas_actuales, metricas_nuevas):

acc_actual = metricas_actuales.accuracy

acc_nuevo = metricas_nuevas.accuracy

SI ESTE.politica ES IGUAL a “any_improvement”:

SI acc_nuevo >= acc_actual:

RETORNAR (Verdadero, “Activado”)

SI NO :

RETORNAR (Falso, “Rechazado”)

SI ESTE.politica ES IGUAL A “min_delta”:

límite_requerido = acc_actual + acc_actual * ESTE.min_delta

SI acc_nuevo >= límite_requerido:

RETORNAR (Verdadero, “Activado”)

SI NO :

RETORNAR (Falso, “Rechazado”)

SI ESTE.politica ES IGUAL A “per_class_f1”

fl_clase_actual = metricas_actuales.fl_scores[ESTE.target_class]

fl_clase_nueva = metricas_nuevas.fl_scores[ESTE.target_class]

SI fl_clase_nueva >= fl_clase_actual:

RETORNAR (Verdadero, “Activado”)

SI NO :

RETORNAR (Falso, “Rechazado”)

4.3. Validación de la Política de Activación Configurable

Tras la evolución del sistema, el historial refleja un control de versiones mucho más granular, donde la activación ya no depende exclusivamente del *accuracy* global, sino de criterios específicos de negocio. Se han registrado intentos de reentrenamiento bajo las políticas per_class_f1 y min_delta, los cuales han resultado en rechazos controlados a pesar de que el modelo presentaba un rendimiento numérico perfecto.

Estos resultados demuestran la eficacia de la clase ActivationPolicy para proteger el entorno de producción. Por ejemplo, se han bloqueado modelos que no aportaban una mejora incremental real (casos de F1-score estancado en 1.0) o que no superaban el umbral de mejora significativa exigido por el parámetro min_delta, garantizando que solo se desplieguen versiones que aporten un valor añadido indiscutible al servicio.